



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

GENERACIÓN DE MALLAS POLIGONALES A PARTIR DE CAVIDADES

INFORME FINAL DE CC6908 PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN COMPUTACIÓN

NICOLÁS ESCOBAR ZARZAR

MODALIDAD:
Memoria

PROFESORA GUÍA:
Nancy Hitschfeld K.

PROFESOR CO-GUÍA:
Sergio Salinas

SANTIAGO DE CHILE
2025

1. Introducción

Las mallas poligonales son estructuras fundamentales en el modelado geométrico y la simulación computacional. Se definen como colecciones de vértices, aristas y caras que, en conjunto, representan la superficie de un objeto, generalmente mediante una subdivisión en triángulos. Esta representación es ampliamente utilizada en áreas como el diseño asistido por computador (CAD), gráficos por computadora, ingeniería estructural, biomecánica y videojuegos. La eficiencia y calidad de las mallas generadas influye directamente en la precisión de las simulaciones y en el rendimiento computacional de los sistemas que las utilizan.

Uno de los problemas recurrentes en este ámbito es la generación automática de mallas poligonales a partir de un conjunto discreto de puntos en el plano, también conocido como problema de triangulación. Si bien existen algoritmos eficientes disponibles en herramientas como Triangle [1] o Detri2 [2] para formar una triangulación de Delaunay [3], la construcción de formas poligonales a partir de dichas triangulaciones sigue siendo un área de investigación activa. En particular, existe un interés creciente por encontrar métodos que generen mallas con mayor fidelidad geométrica, adaptabilidad local, y que mantengan cualidades deseables como ángulos adecuados y buena distribución de los elementos.

Una triangulación de Delaunay se define como tal si cumple con la propiedad de que para todos los triángulos de la triangulación, se cumple que el circuncírculo del triángulo solo contiene a los vértices de su triángulo respectivo y no los vértices de cualquier otro (ver Figura 1). Estas son de particular importancia porque maximizan el tamaño del ángulo más pequeño de la triangulación. Esta propiedad previene problemas de precisión que ocurren al tener ángulos muy agudos los cuales propician “casos degenerados” donde los puntos de un triángulo son interpretados como colineales lo cual puede provocar cálculos erróneos e incluso que los programas que trabajen con las mallas resultantes que no sean lo suficientemente robustos fallen por completo en el peor de los casos.

Este trabajo de memoria se enfoca en el desarrollo y análisis de una estrategia alternativa de generación de mallas en dos dimensiones, basada en el concepto de **cavidad** o *concavity* como se describe en el artículo Triangle [1]. La idea central es que, a partir de una triangulación de Delaunay (como la de la Figura 1), se seleccionan ciertos triángulos utilizando un criterio definido por el usuario. Luego, se calculan los circuncentros p_i de estos triángulos, y se identifican los conjuntos de triángulos de la malla cuyo circuncírculo contiene alguno de estos puntos por separado. La unión de las aristas del borde de estos triángulos vecinos para un punto p forma una **cavidad**, la cual define un nuevo polígono. Este procedimiento permite generar regiones poligonales que pueden servir como base para construir un nuevo tipo de mallas poligonales.

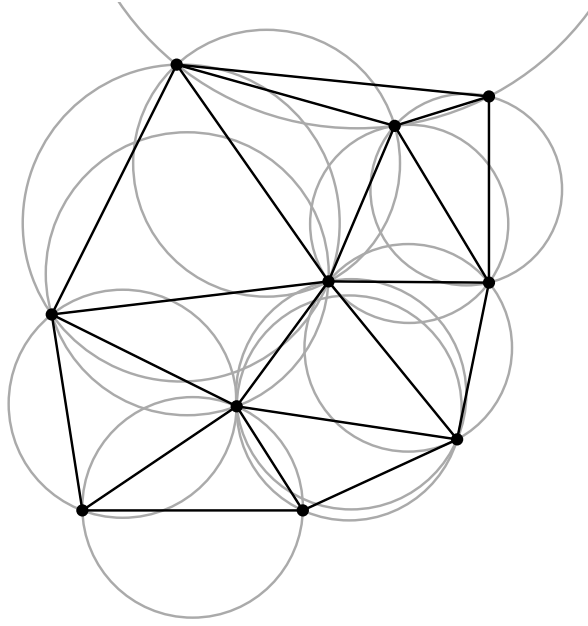


Figura 1: Triangulación de Delaunay con circuncírculos compuesta por 10 vértices.

A modo ilustrativo, en la Figura 2 se presenta una selección de triángulos a partir de la triangulación de la Figura 1.

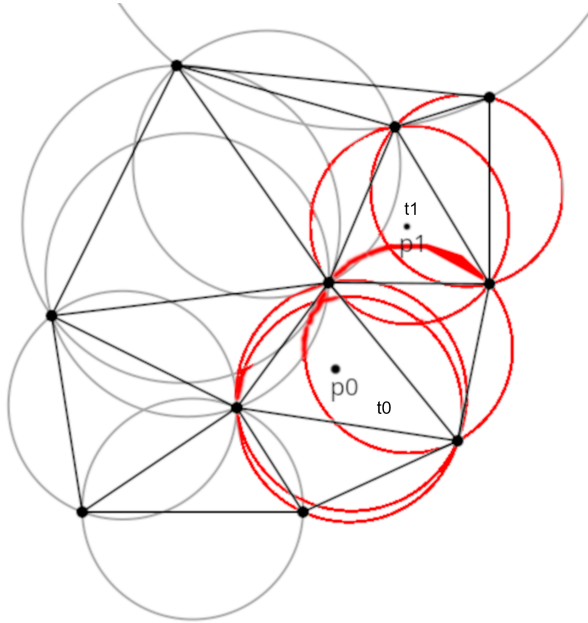


Figura 2: Selección de triángulos para formar un polígono a partir de una cavidad.

En este ejemplo, se seleccionaron dos triángulos t_0 y t_1 , cuyos respectivos circuncentros están representados por los puntos p_0 y p_1 . Estos puntos están contenidos en los circuncírculos de varios triángulos vecinos, incluyendo aquellos que los generaron. La unión de las aristas de borde de los triángulos que contienen a p_0 y la unión de las aristas de borde

de los triángulos que contienen a p_1 forman cada uno una cavidad, las cuales se pueden ver en la Figura 3.

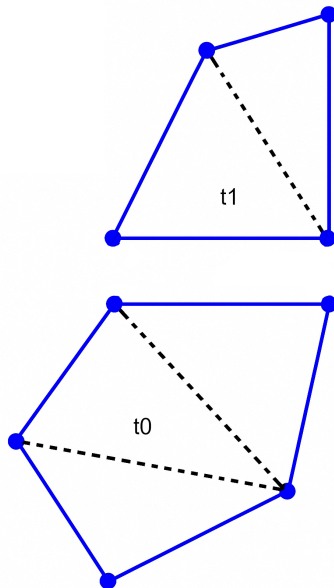


Figura 3: Polígonos resultantes de las cavidades marcados en azul con los lados discontinuos borrados

Este enfoque mediante cavidades no solo busca explorar una nueva forma de construir mallas, sino también comparar su rendimiento y características con métodos existentes. En particular, se contrastará esta técnica con el generador de mallas ***Polylla*** [4], un sistema moderno que emplea estructuras de datos avanzadas para lograr eficiencia y calidad en la generación de polígonos.

La necesidad de desarrollar nuevas estrategias para la generación de mallas responde a múltiples factores: mejorar la eficiencia de los algoritmos existentes, generar elementos con mejores propiedades geométricas, y adaptar las mallas a dominios complejos sin intervención manual.

2. Situación Actual

Existen muchos algoritmos para generación de mallas poligonales, siendo los más relevantes para este trabajo Triangle [1], Detri2 [2] y Polylla [4].

2.1. Triangle

El algoritmo Triangle [1] se basa en el algoritmo de Ruppert [5] y consta de 4 etapas, de las cuales las primeras 2 son exactamente las mismas que las de Ruppert, estas involucran triangular un *planar straight line graph* (o PSLG), un conjunto de vértices y segmentos que describen un polígono como el de la Figura 4, y posteriormente insertar los segmentos originales de la triangulación como se ve en la Figura 5 y la Figura 6.

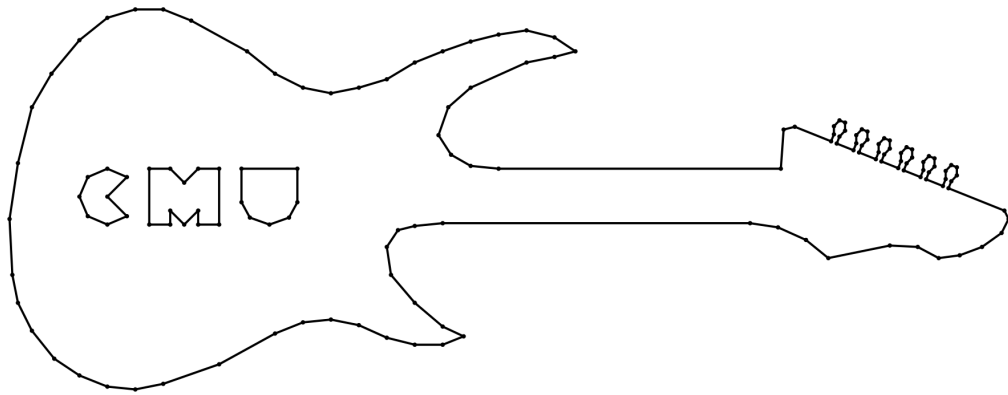


Figura 4: PSLG de entrada una guitarra electrica como se ilustra en Triangle [1]

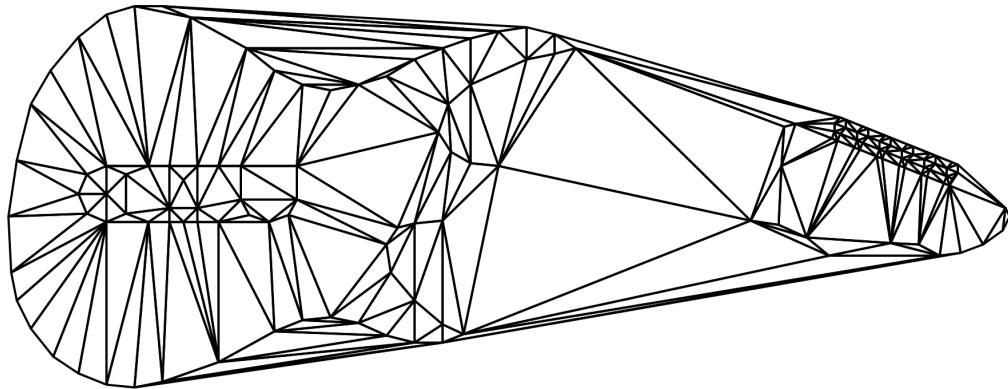


Figura 5: Triangulación del PSLG de la Figura 4 con segmentos originales faltantes [1]

Dado que esta malla no describe precisamente al polígono original del PLSG, la segunda etapa consta de reintroducir los segmentos originales del PLSG, esto se puede hacer de 2 maneras posibles según preferencia del usuario. La primera es insertar un nuevo vértice que corresponda al punto medio de alguno de los segmentos que no aparezcan en la triangulación anterior y usar el algoritmo incremental de Lawson [6] para obtener una nueva triangulación de Delaunay basada en la anterior con el vértice adicional. Esto genera que el segmento original se divida en 2 y la nueva triangulación podría tener el segmento original formado por estos 2 sub segmentos. En caso de que los sub segmentos aún no formen un arco en la triangulación, el proceso de insertar un vértice del punto medio se repite recursivamente para los sub segmentos hasta que el segmento original exista como una secuencia de segmentos lineales. La manera alternativa de insertar los segmentos originales, y la que se utiliza por defecto, es convertir la triangulación a una triangulación de Delaunay restringida, en la cual los segmentos originales deben aparecer. Esto se logra eliminando los triángulos que intersecan el segmento que se desea agregar y luego re triangulando las regiones a cada lado del segmento insertado, resultando en la Figura 6.

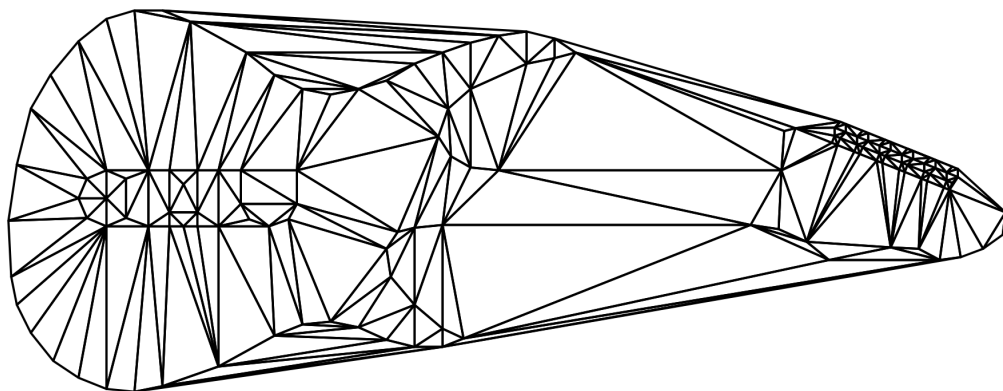


Figura 6: Triangulación restringida del PSLG de la Figura 4 [1]

La tercera etapa difiere del algoritmo de Ruppert y consiste en remover los triángulos extra que están fuera del borde definido por el PSLG original, como aquellos presentes en zonas que originalmente eran no convexas, a las cuales Shewchuk llama concavidades, y los presentes en agujeros (como los del interior del cuerpo de la guitarra en el ejemplo). Esto se ilustra en la Figura 7.

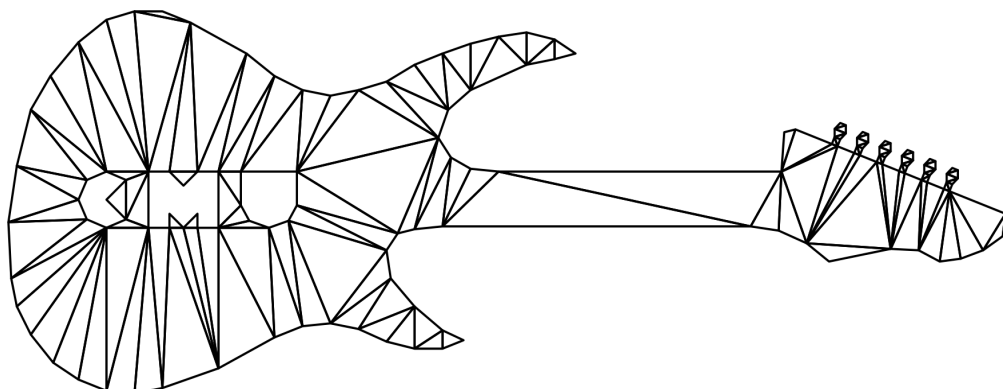


Figura 7: Triangulación restringida del PSLG de la Figura 4 con triángulos extra removidos [1]

La última etapa del algoritmo consiste en el refinamiento de la malla insertando vértices y re triangulando con el algoritmo incremental de Lawson [6] hasta que las restricciones de ángulo mínimo y área máxima de triángulo definidas por el usuario se cumplan. Esta inserción se hace siguiendo 2 reglas, la regla de *segmentos encerrados* y la de los *triángulos malos* dándole siempre prioridad a la primera:

- El *círculo diametral* de un segmento es el círculo único más pequeño que contiene el segmento como su diámetro. Un segmento se dice que está *encerrado* si un punto que no es un extremo del segmento está dentro de su círculo diametral. Cualquier segmento encerrado que aparezca se separa insertando un vértice en su punto medio. Los dos sub segmentos resultantes tienen círculos diametrales más pequeños y podrían estar o no estar encerrados. El proceso se repite hasta que no queden segmentos encerrados como se ve en la Figura 8.

- El *circuncírculo* de un triángulo es el círculo único cuya circunferencia pasa por todos los vértices del triángulo. Un triángulo se dice que es *malo* dependiendo de algún criterio, por ejemplo si tiene un ángulo que es muy pequeño o un área que es muy grande para satisfacer las restricciones impuestas por el usuario. Un triángulo malo se destruye insertando un vértice en su *circuncentro* (el centro de su circuncírculo). Está asegurado que el triángulo malo será eliminado como se ve en la Figura 9 para mantener la propiedad de Delaunay. Si el vértice insertado encierra un segmento (como se define en la regla del círculo diametral), este será removido deshaciendo la inserción y los segmentos que encerraba se separarán según la regla del círculo diametral.

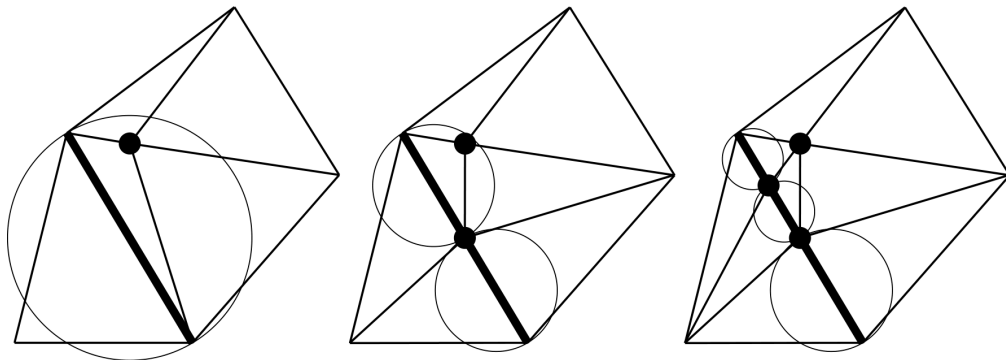


Figura 8: Segmentos divididos según su círculo diametral [1]

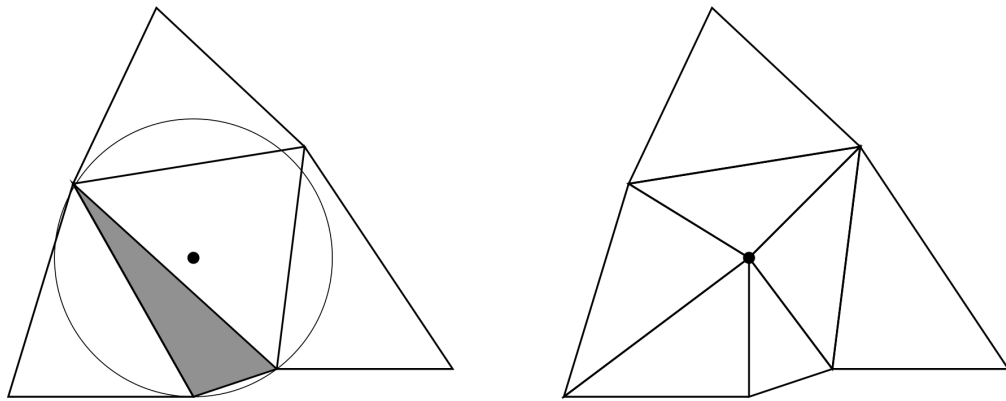


Figura 9: Triángulo separado según su circuncentro [1]

Para el ejemplo de la Figura 4, la malla resultante generada por Triangle es la que se ilustra en la Figura 10.

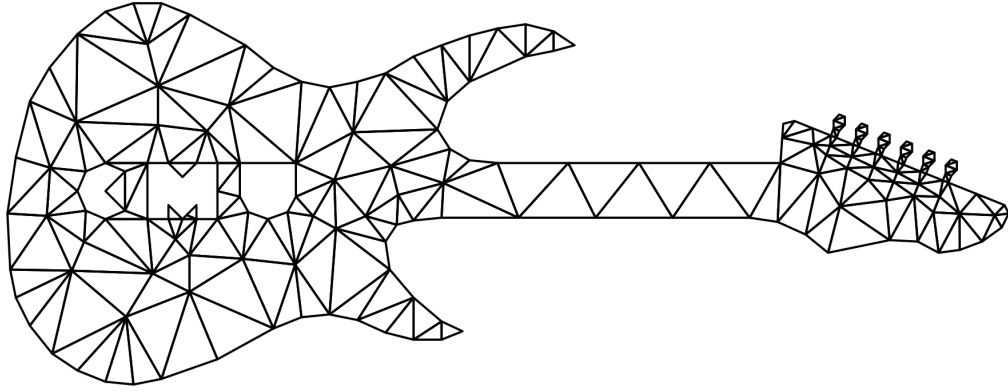


Figura 10: Malla poligonal final resultante de aplicar el algoritmo Triangle [1]

Esta última parte del algoritmo es de particular importancia, ya que el criterio del circuncírculo es análogo al de la cavidad, sin embargo, en este caso solo se utiliza para refinar la malla y retriangular, las cavidades. Este trabajo de memoria busca explorar más a fondo este proceso y utilizar las cavidades para generar mallas de polígonos generales.

2.2. Detri2

El software Detri2[2] permite generar triangulaciones a partir de nubes de vértices aleatorios, y utilizar distintos criterios para refinar la triangulación a través de una interfaz gráfica que permite una gran variedad de opciones y resulta muy útil para generar o verificar geometrías. Además de la triangulación de un conjunto de puntos, Detri2 también permite visualizar su diagrama de Voronoi. El diagrama de Voronoi es una partición de un dominio P en regiones o “celdas” R_i , de las cuales cada una contiene un punto p_i llamado “semilla” y los puntos q contenidos en R_i cumplen que $\|q - p_i\| < \|q - p_j\| \forall p_i, p_j \in P, i \neq j$ donde p_j es la semilla de cualquier otra celda distinta a R_i . El diagrama de Voronoi también se le conoce como el *dual* de una triangulación de Delaunay, esto se debe a que, dada una triangulación de Delaunay, se puede obtener su diagrama de Voronoi equivalente si los circuncentros de los triángulos se convierten en puntos para las regiones de Voronoi. Cada punto (o sitio) en la triangulación es una «semilla» del diagrama de Voronoi.

En la Figura 11 y la Figura 12 se puede ver una triangulación de Delaunay, su diagrama de Voronoi equivalente, y ambos superpuestos. La Figura 12 muestra un ejemplo hecho en Detri2.

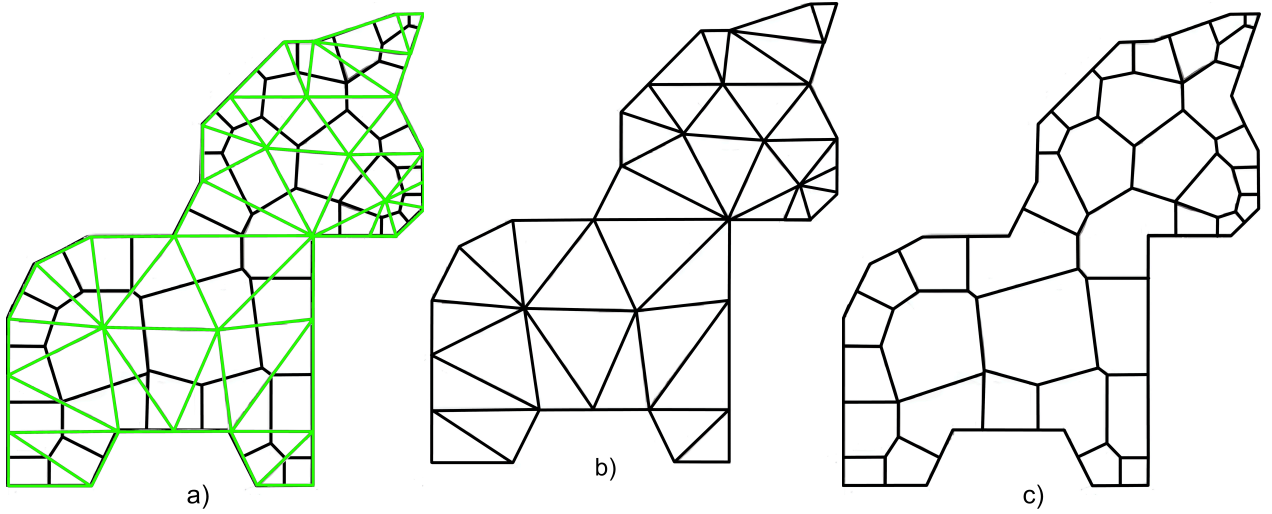


Figura 11: a) Triangulación de Delaunay y Diagrama de Voronoi superpuesto,
b) Triangulación de Delaunay,
c) Diagrama de Voronoi [4]

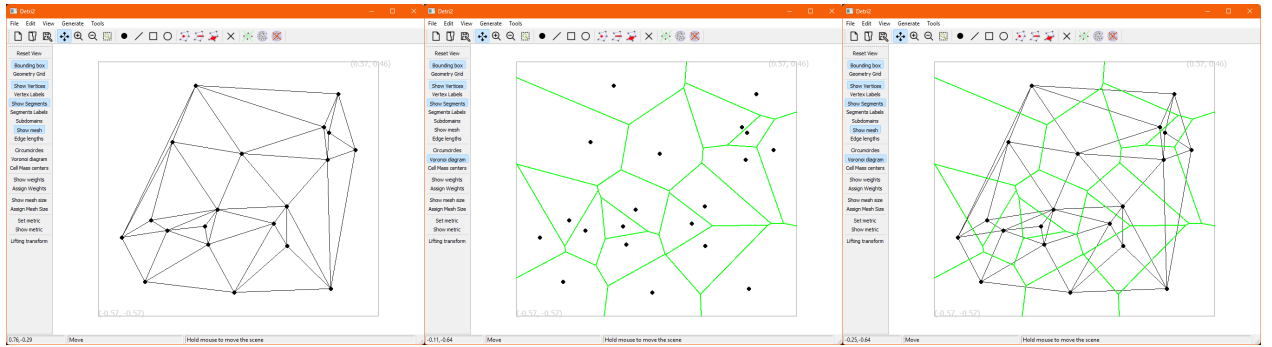


Figura 12: Triangulación de Delaunay y su Diagrama de Voronoi dual en el software Detri2

2.3. Polylla

Por otro lado, el algoritmo Polylla, busca generar una malla poligonal a partir de una triangulación arbitraria, usando lo que denomina como *Terminal-edge regions* o regiones de arista terminal, definidas según el *longest edge propagation path* (camino de propagación de arista más larga o *Lepp* [7]) de los triángulos, las cuales utiliza para generar una partición de la triangulación que se asemeja a un diagrama de Voronoi [8].

El *Lepp* o camino de propagación de arista más larga de un triángulo se define de la siguiente manera: Por cada triángulo t_i en cualquier triangulación Ω , el $Lepp(t_i)$ es la lista ordenada de todos los triángulos $t_0, t_1, t_2, \dots, t_{l-1}, t_l$ con $l \in \mathbb{N}$, tal que t_i es el triángulo vecino de t_{i-1} a través de la arista más larga de t_{i-1} , para $i = 1, 2, \dots, l$. Si una arista más larga es compartida por t_{l-1} y t_l esta se define como una arista terminal donde termina el Lepp y t_{l-1} y t_l son triángulos terminales. Una región de arista terminal se define como la unión de los triángulos t tal que $Lepp(t)$ termina en la misma arista terminal. En la Figura 13 se puede ver un ejemplo de región de arista terminal.

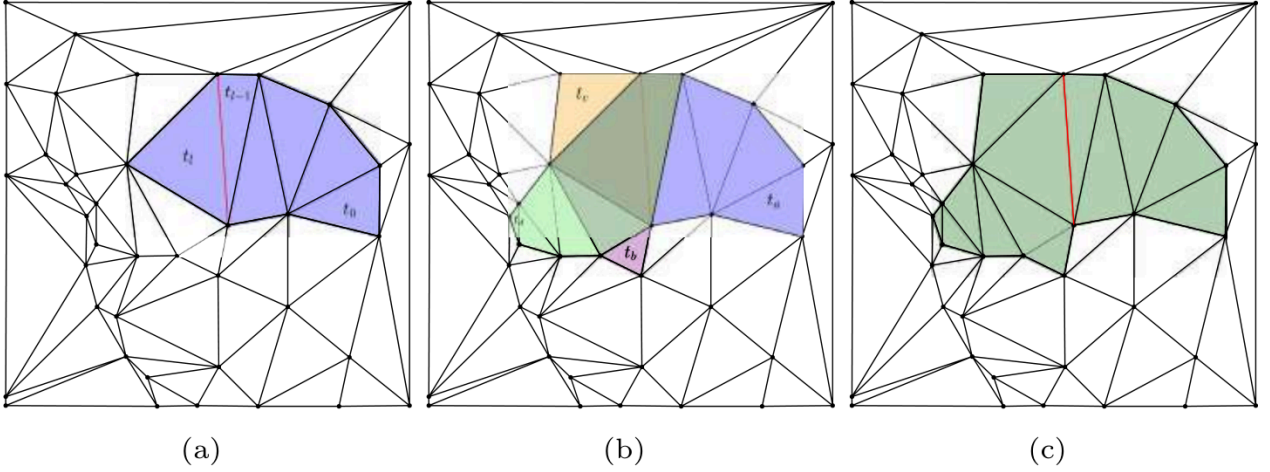


Figura 13: Región de arista terminal. a) $Lepp(t_0)$ donde la arista roja es la arista terminal. b) Cuatro Lepps con la misma arista terminal: $Lepp(t_a)$, $Lepp(t_b)$, $Lepp(t_c)$, $Lepp(t_d)$. c) Región de arista terminal generada por la unión de los Lepp de b) [4]

Además de las aristas terminales, Polylla [4] define los siguientes tipos de aristas. Dada una arista e y dos triángulos t_1 y t_2 que comparten e :

- *Frontier-edge* o Arista frontera: e no es la arista más larga ni de t_1 ni de t_2 .
- *Internal-edge* o Arista interna: e es la arista más larga de t_1 , pero no de t_2 o viceversa.
- *Boundary edge* o Arista de borde: e pertenece a un solo triángulo. Se manejan como aristas frontera.
- *Barrier edge* o Arista barrera: Arista frontera que queda adentro de una región terminal (y no es el borde).

Polylla consiste de tres fases: Primero etiqueta las aristas de la triangulación de entrada según las categorías anteriores para formar regiones terminales y además designa un *triángulo semilla* en cada región de arista terminal para construir las regiones. Luego, a partir de cada triángulo semilla, hace un recorrido en sentido antihorario o *counter clockwise* (CCW en inglés) de la región de arista terminal para encontrar aristas frontera las cuales formaran la región. Algunas regiones pueden terminar como polígonos no simples, es decir, que tienen puntos colineales o aristas que se intersecan entre sí, por lo que hace una fase de reparación donde las aristas barrera se particionan en polígonos simples. En la Figura 14 se puede ver una partición de un polígono generada por regiones de arista terminal que presenta una región con un polígono no simple en verde.

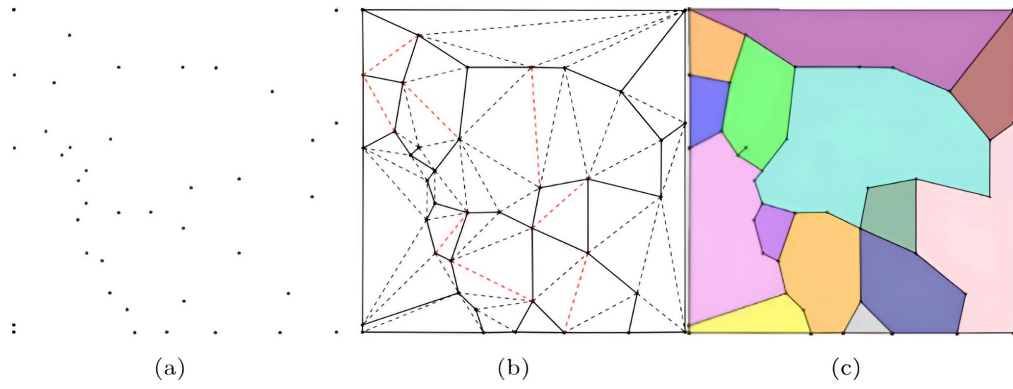


Figura 14: a) Colección de vértices aleatorios, b) Triangulación de Delaunay donde las líneas sólidas son aristas frontera, las líneas punteadas negras son aristas internas y las aristas punteadas rojas son aristas terminales. c) Partición a partir de regiones de arista terminal [4]

En la Figura 15 se puede ver una triangulación de Delaunay y la malla generada por Polylla a partir de ella.

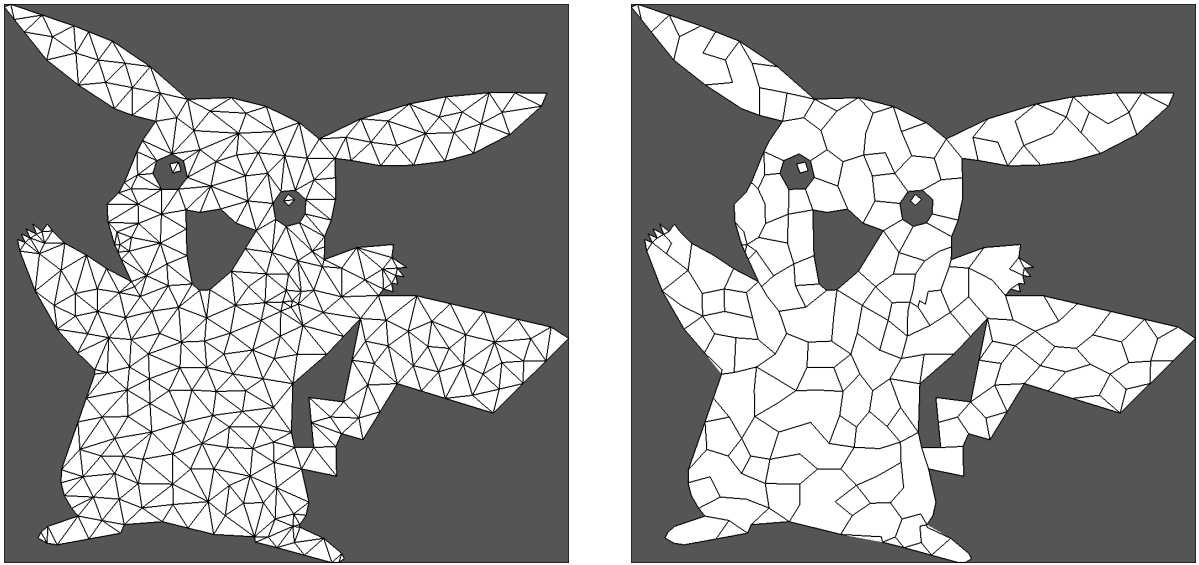


Figura 15: Triangulación de Delaunay y su malla Polylla respectiva [9]

El algoritmo Polylla destaca por sobre otros algoritmos debido a su gran simplicidad y eficiencia en comparación a algoritmos convencionales de construcción de diagramas de Voronoi restringidos, ya que toma bastante menos tiempo en construir con una cantidad de polígonos 3 veces menor y la mitad de vértices que una malla poligonal de un diagrama de Voronoi hecho a partir de la misma triangulación. A esto se le suma también la utilidad de las mallas Polylla en simulaciones de diversos fenómenos que hacen uso del *Virtual Element Method* (VEM) [10] tales como mecánica computacional, dinámica de fluidos, propagación de ondas entre otros problemas que requieren soluciones numéricas de ecuaciones diferenciales parciales. El algoritmo de construcción de mallas basado en cavidades se muestra

como una alternativa a Polylla y mallas basadas en el diagrama de Voronoi que permite explorar el alcance de una nueva estrategia, sus propiedades generativas y de escalabilidad.

3. Objetivos

3.1. Objetivo General

Diseñar e implementar un algoritmo que permita generar polígonos a partir de una *cavidad* desde una triangulación de Delaunay y comparar su desempeño con el generador de mallas Polylla en cuanto a tiempo, memoria, calidad de las mallas, número de vértices, aptitud para el VEM, entre otros criterios.

3.2. Objetivos Específicos

1. Investigar como funciona el algoritmo de mallas Polylla, sus estructuras de datos y métricas apropiadas para el VEM.
2. Definir escenarios de prueba para que el conjunto de polígonos que conforman la cavidad computada sean los correctos y generar un nuevo polígono a partir de ellos.
3. Comparar el desempeño del algoritmo basado en cavidades con el algoritmo de Polylla basado en regiones terminales en cuanto a eficiencia en tiempo y memoria.
4. Diseñar e implementar estrategias para seleccionar triángulos cuyo circuncentro se usará para definir cavidades.
5. Comparar la calidad de las mallas resultantes de ambos algoritmos bajo diversas métricas como calidad de la malla, número de vértices, cantidad de polígonos, etc.

3.3. Evaluación

Para evaluar este trabajo es necesario comparar cualitativamente la malla poligonal resultante del algoritmo basado en cavidades con el algoritmo de Polylla. Se deben evaluar criterios como:

1. La correctitud de la malla poligonal generada. ¿Se ajusta realmente al objeto que se desea modelar?
2. La cantidad de triángulos que componen una cavidad en comparación a la cantidad de triángulos que forman los polígonos en Polylla. ¿Usa igual o menos polígonos?
3. Los ángulos internos de los polígonos generados, el grado de convexidad.
4. El tiempo de ejecución. ¿Es igual o más eficiente el algoritmo basado en cavidades para generar una malla a partir de la misma triangulación que Polylla?
5. El costo en espacio. ¿Utiliza igual o menos memoria que Polylla para generar la misma malla?
6. La calidad de las mallas. ¿La malla generada por el algoritmo basado en cavidades cumple propiedades deseables sobre polígonos? ¿Es útil para simulaciones?

Para verificar lo anterior se hará uso de las mallas generadas a partir de cavidades para encontrar una solución numérica de una ecuación de *Poisson* en dos dimensiones en un dominio cuadrado con condiciones de borde haciendo uso del VEM[10], tal como se hace con Polylla[11, 12].

4. Solución Propuesta

La solución propuesta involucra adoptar las estructuras de datos de Polylla[9] escritas en C++ y aplicarlas a este problema para generar una malla de manera eficiente. Son de especial interés 2 estructuras de datos fundamentales, la estructura `vertex` y la estructura `halfEdge`[13, 14].

La estructura `vertex` (Código 1) define las coordenadas `x` e `y` de los puntos de la malla poligonal final, además clasifica a los puntos según si son parte del borde del dominio o no, lo cual será útil al momento de representar la cápsula convexa de la malla final. También se encarga de guardar cuál `halfEdge` incide en este vértice.

La estructura `halfEdge` (Código 2) representa las aristas del polígono de una forma particular, no solo es el segmento que une un vértice v_i con otro vértice v_j , sino que también guarda información relevante a la hora de recorrer el polígono, como por ejemplo, cuál es su vértice de origen (la “cola” del `halfEdge`), cuál es el siguiente `halfEdge` del triángulo (o cara) actual en orientación CCW, el `halfEdge` anterior de este triángulo y además cuál es su `halfEdge` `twin`, es decir, el `halfEdge` que une a los mismos dos vértices en CCW pero desde la perspectiva del triángulo vecino. Al igual que los vértices, también guarda la información de si está en el borde o no. Notar que esta estructura de datos permite representar mallas de cualquier tipo de celdas poligonales, no solo triángulos.

Se puede ver una representación gráfica de estas estructuras en la Figura 16.

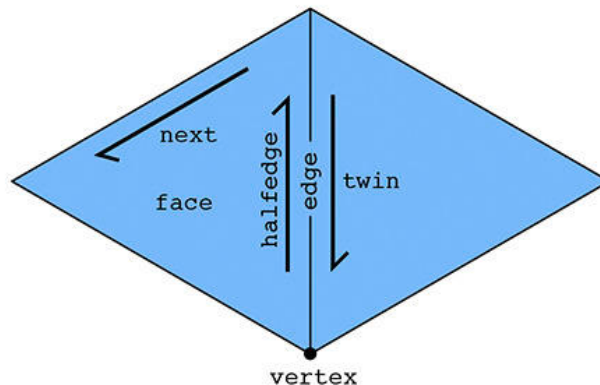


Figura 16: Representación de estructuras `vertex` y `halfEdge`

```
struct vertex {  
    double x;  
    double y;  
    bool is_border = false;  
    int incident_halfedge; // <- indice a un array de halfedges  
};
```

Código 1: Estructura `vertex`

```
// Todos sus atributos son índices al objeto relevante
struct halfEdge {
    int origin;
    int twin;
    int next;
    int prev;
    int is_border;
};
```

Código 2: Estructura halfEdge

Teniendo estas estructuras de datos se procederá de la manera siguiente: Similar a Polylla, se recibirá como entrada una triangulación de Delaunay en un conjunto de 3 archivos, un archivo `.node` con los vértices y marcador de borde, un archivo `.ele` con los triángulos de las triangulaciones (qué vértices forman cuáles triángulos) y un archivo `.neigh` con las listas de adyacencia de cada triángulo (sus vecinos). A partir de estos archivos, se creará un objeto `Triangulation`[9] encargado de almacenar estos datos en listas de `vertex` y `halfEdge` en vectores de C++.

Utilizando algún criterio a definir según experimentación, tales como, triángulos que cumplen o no cumplen ciertas restricciones de ángulos o de área, se seleccionará un subconjunto de los triángulos recibidos y se les calculará su circuncentro p_i y radio r_i usando un método derivado de determinantes [15]. La forma en que estos criterios serán aplicados, se hará con estructuras simples que definen el método `operator()` y retornan `bool`, (también llamados funtores) resultando en algo similar al patrón *composite*, para formar criterios de selección arbitrariamente complejos los cuales serán entregados al refinador, luego el refinador verificará cada triángulo con estos criterios. Se opta por el uso de funtores, porque estos proveen una api común (el método `operator()`) y pueden definirse con cualquier atributo que el criterio necesite, como un ángulo mínimo, un área mínima, etc.

Para determinar el circuncentro de los triángulos se procederá de la manera siguiente: sean A , B y C los vértices de un triángulo en orientación CCW, primero, para simplificar cálculos y sin pérdida de generalidad, se aplica una traslación a estos vértices de modo que A , B o C quede en el origen, por simplicidad, se asumirá que A se traslada al origen y se definen los siguientes nuevos vértices:

$$A' = A - A = (0, 0)$$

$$B' = B - A$$

$$C' = C - A$$

También se computará un valor D que corresponde al cuádruple del área del triángulo desplazado:

$$D = 2[(A' \times B')_z + (B' \times C')_z + (C' \times A')_z]$$

$$D = 2(B' \times C')_z$$

$$D = 2(B'_x C'_y - B'_y C'_x)$$

Luego las coordenadas del circuncentro desplazado U' serán

$$U'_x = \frac{1}{D} [C'_y (B'^2_x + B'^2_y) - B'_y (C'^2_x + C'^2_y)]$$

$$U'_y = \frac{1}{D} [B'_x (C'^2_x + C'^2_y) - C'_x (B'^2_x + B'^2_y)]$$

Se debe tener precaución al calcular D , ya que este podría ser cero, indicando la presencia de un “caso degenerado”, pero estos triángulos también serán eliminados como parte de la cavidad. Esto podría ocurrir por errores de precisión en los datos.

Finalmente, las coordenadas del circuncentro real estarán ubicadas en:

$$U = U' + A$$

Conociendo los circuncentros, estos se almacenan en un vector de tuplas (p_i, t_i) donde p_i es el circuncentro y t_i es el índice del triángulo al cual pertenece este circuncentro. Se guardan ambos valores para revisar los triángulos vecinos y los vecinos de estos, ya que por construcción, el circuncírculo de un triángulo más lejano que los vecinos segundos, no puede contener a p_i . Luego se creará un *hash map* donde las llaves serán las coordenadas de los circuncentros (vértices) y los valores serán vectores de índices para almacenar los triángulos que forman parte de cada cavidad. Para cada uno de los p_i se verifica si sus triángulos vecinos (y vecinos segundos) lo contienen en su circuncírculo. Esta verificación se hará mediante el *incircleTest* de Shewchuk[16], este también hace uso de un método basado en determinantes. Una vez computadas las cavidades, se mantendrán solo las aristas de borde para generar nuevos polígonos. Finalmente, se retornan los polígonos resultantes del proceso anterior contenidos en el *hash map* en un nuevo objeto de malla que puede ser guardado en formatos como `.off`, o `.node`, `.ele` y `.neigh`.

5. Plan de Trabajo

1. Estudiar a cabalidad Polylla para utilizar sus clases y estructuras de datos para la generación de mallas a partir de cavidades. [abril-mayo]
2. Diseñar y programar la implementación del método para formar la cavidad a nivel de código como extensión de Polylla [mayo-julio] (Trabajo a adelantar hasta aquí)
3. Probar distintos criterios de selección de triángulos y guardar el registro de los resultados a medida que se escribe el informe final [agosto-septiembre].
4. Seleccionar los mejores métodos de elección de triángulos, comparar su desempeño con la generación de mallas de Polylla y registrar los datos en el informe final [septiembre-octubre].
5. Comparar el rendimiento de las mallas generadas en otras aplicaciones frente a la misma malla generada por Polylla [noviembre-diciembre].

	Semana														
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Ajustes de código															
Hacer refactor a polylla															
Integrar el código de cavidades															
Pruebas de criterios															
Probar criterios de selección															
Comparación con polylla															
Diseñar test comparativo															
Correr test comparativo															
Comparación con otras apps															
Elegir aplicaciones de mallas															
Diseñar test comparativo															
Correr test comparativo															
Escribir la memoria															
Desarrollo del documento															
Registro de resultados															
Análisis de resultados															

6. Trabajo Adelantado

Tras el estudio del código de Polylla[9], se detectaron varios problemas de diseño que hacen el código difícil de extender para otros usos dada su alta especialización en pos de la eficiencia, por lo tanto, se diseñó una nueva estructura del código que siga principios

[illegible]

Este diseño introduce más flexibilidad al separar la clase `Triangulation` en diversas clases con propósitos específicos cada una.

La clase principal en esta estructura (la API expuesta al usuario para realizar operaciones) es la clase **PolygonalMesh** (Figura 18), esta clase cumple la función de integrar las otras clases en una interfaz única en común que permite diversos escenarios de configuración según el tipo de archivo de malla a leer o escribir, el algoritmo de refinamiento a utilizar y el tipo de malla subyacente.

[MeshData] PolygonalMesh
- data: MeshData* - refinedData: MeshData* - reader: MeshReader - writer: MeshWriter - refiner: MeshRefiner
+ setReader(reader: MeshReader): PolygonalMesh + setRefiner(refiner: MeshRefiner): PolygonalMesh + readMeshFromFile(filepath: String): PolygonalMesh + refineMesh(): PolygonalMesh + writeStatsToJson(filepath: string): void + getOriginalMeshData(): MeshImpl + getRefinedMeshData(): MeshImpl + getRefinementStats(): unordered_map<MeshStat,int> + getRefinementTimes(): unordered_map<TimeStat,double> + PolygonalMesh(reader: MeshReader): PolygonalMesh + PolygonalMesh(reader: MeshReader, writer: MeshWriter): PolygonalMesh

Figura 18: Clase principal **PolygonalMesh** como interfaz común para los otros componentes del código

El almacenamiento de los datos reales de la malla, es decir, sus vértices, aristas y caras se almacenan en alguna clase que implemente **MeshData** (Figura 19), las cuales saben que estructura de datos tendrán que manejar mediante el uso de *concepts* [18, 19], que se asemeja a las *interfaces* de Java, en el sentido de que solo definen los métodos disponibles y su firma, pero no la implementación, con la diferencia de que los *concepts* no definen un tipo real y solo existen para ser usados como restricciones para tipos de *template* de C++ en la declaración de las clases, métodos u atributos. Estos *templates* permiten generalizar

tipos, haciendo posible la extensión futura con otros tipos de malla de forma simple casi sin modificaciones al código existente. Para asegurar correctitud, el uso de tipos explícitos se declara en el *header* de las clases que utilicen un tipo *template* que requiera de un *concept* específico, así es posible corroborar en tiempo de compilación (o incluso antes con el apoyo de herramientas de análisis estático de código) si algún tipo nuevo se adhiere correctamente al *concept* y si es que no, la razón de por qué. Algunas de estas razones podrían ser que la firma de un método no coincide, o le falta algún atributo a la clase, en raros casos puede arrojar errores crípticos que parecen no tener sentido, como por ejemplo que el argumento de un *template* es algún tipo primitivo como *int* que no posee métodos, pero esto suele ser una señal de uso de dependencias circulares o similar.

[Mesh] «concept» MeshData
+ getVertex(int vertexIndex): Mesh::VertexType& + getEdge(int edgeIndex): Mesh::EdgeType& + getPolygon(int polygonIndex): int + numberOfVertices(): size_t + numberOfEdges(): size_t + numberOfPolygons(): size_t + updatePolygonCount(int amount): void + updateVertexCount(int amount): void + updateEdgeCount(int amount): void + getNeighbors(int polygonIndex): vector<int> + getVerticesOfTriangle(polygonIndex: int, v0: Vertex&, v1: Vertex&, v2: Vertex&): void + Mesh(vertices: vector<Mesh::VertexType>, edges: vector<Mesh::EdgeType>, faces: vector<int>) + Mesh(mesh: const Mesh&)

Figura 19: *Concept* para declarar requisitos que una clase debe cumplir para ser considerada una malla valida al usarse como *template*

El *concept* **MeshData**, además de definir métodos, también impone que cada malla debe definir un alias de tipo para sus vértices y aristas, y también para sus índices. Forzar la restricción de alias de índices es solo para darle más claridad al código, pero el alias de tipo para los vértices y aristas permite más genericidad a la hora de utilizarlos en métodos que reciban algún **MeshData** como *template*, de esta forma, si los vértices y aristas de distintos tipos de malla realizan operaciones comunes, también se les puede adjuntar un *concept* y como el alias siempre se refiere al tipo concreto explícitamente por virtud de ser parte del *template*, nunca hay ambigüedad sobre qué método se llama, aumentando en gran medida la eficiencia en contraste a una jerarquía clásica de herencia de clases virtuales como interfaces que deben hacer resolución de métodos en tiempo de ejecución.

Actualmente hay solo un tipo de malla que se adhiere a este *concept*: la malla basada en *half edges* de la clase `HalfEdgeMesh` (Figura 20). Esta clase toma gran parte de lo que es la clase `Triangulation` de Polylla, en particular, los métodos para recorrer la malla y construirla. Esta clase contiene 3 vectores clave, el vector de vértices, el vector de half edges y el vector llamado `polygons` (o `faces` en Polylla), cuya función es almacenar tripletes de vertices consecutivos que forman triángulos, es decir, cada índice múltiplo de 3 es un vértice de un triángulo distinto.

HalfEdgeMesh
- vertices: vector<HEVertex> - halfEdges: vector<HalfEdge> - polygons: vector<int>
+ getEdge(edge: int): HalfEdge + getVertex(vertex: int): HEVertex + next(int edge): int + origin(int edge): int + target(int edge): int + twin(int edge): int + prev(int edge): int + edgeLength2(int edge): double + ccwEdgeToVertex(int edge): int + cwEdgeToVertex(int edge): int + isBorderFace(int edge): int + isInteriorFace(int edge): int - constructInteriorHalfEdgesFromFacesAndNeighs(faces: std::vector<int>, neighbors: std::vector<int>): void - constructInteriorHalfEdgesFromFaces(faces: std::vector<int>): void - constructExteriorHalfEdges(): void + HalfEdgeMesh(vertices: vector<HEVertex>, edges: vector<HalfEdge>, faces: vector<int>): HalfEdgeMesh + HalfEdgeMesh(vertices: vector<HEVertex>, edges: vector<HalfEdge>, faces: vector<int>, neighbors: vector<int>): HalfEdgeMesh

Figura 20: Clase que modela una malla con aristas `HalfEdge`

La clase `HalfEdgeMesh` hace uso de los *structs* `HEVertex` y `HalfEdge` (Figura 21), que modelan vértices (con atributos extra para mallas *half edge*) y *half edges*. Este *struct* para vértices tiene muchas más capacidades que la estructura `vertex` de Polylla, más específicamente, define distintos operadores junto con otros vértices y operaciones vectoriales como producto punto `dot` y `cross2d`, siendo este último esencial a la hora de comprobar una correcta orientación de las aristas. Los métodos estáticos `inCircle` y `findCircumcenter` corresponden a funciones que implementan el `inCircleTest` de Triangle [20] y el cálculo del circuncentro basado en determinantes [15].

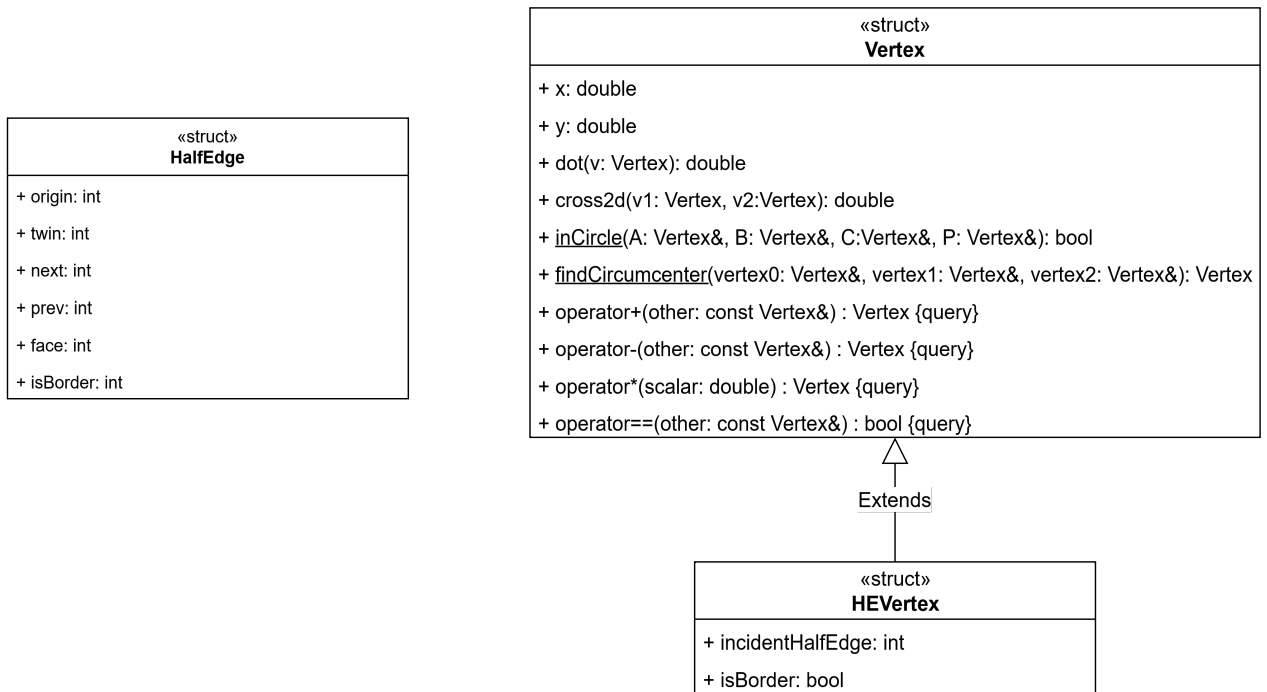


Figura 21: Estructuras básicas que modelan vértices y *half edges*

La lógica para leer y escribir mallas se abstrae en las clases que implementan **MeshReader** y **MeshWriter**, donde cada implementación se encarga de un tipo de archivo concreto (Figura 22 y Figura 23).

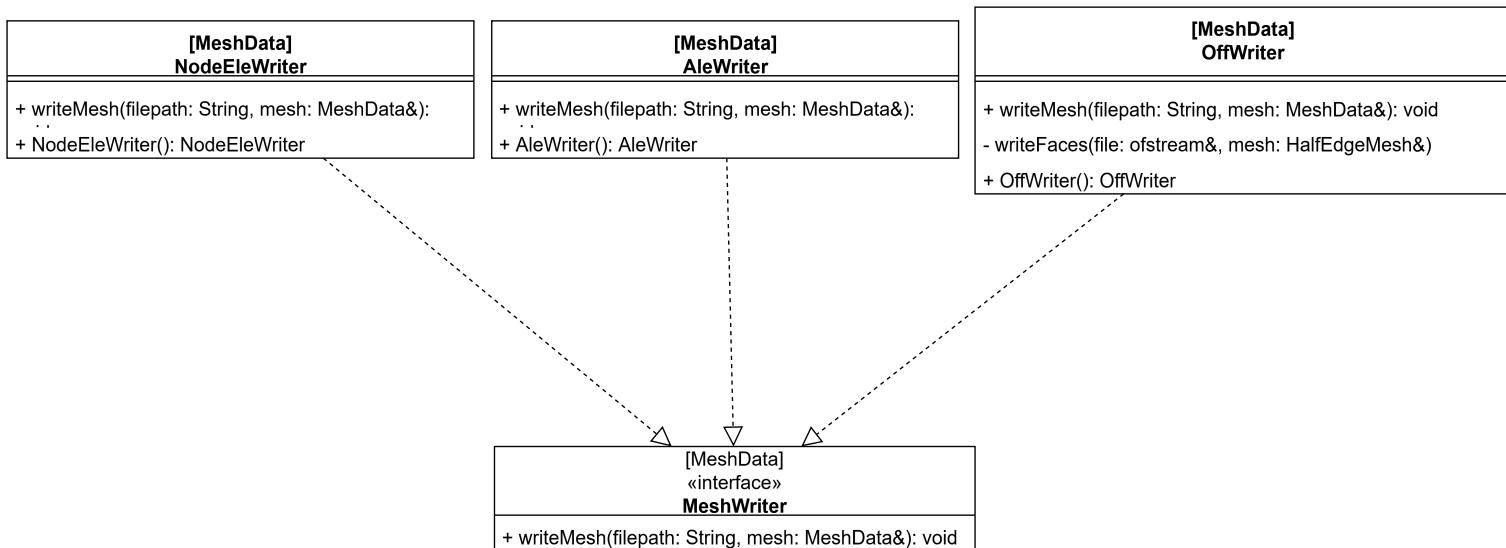


Figura 22: Interfaz y clases para escribir la malla a archivos

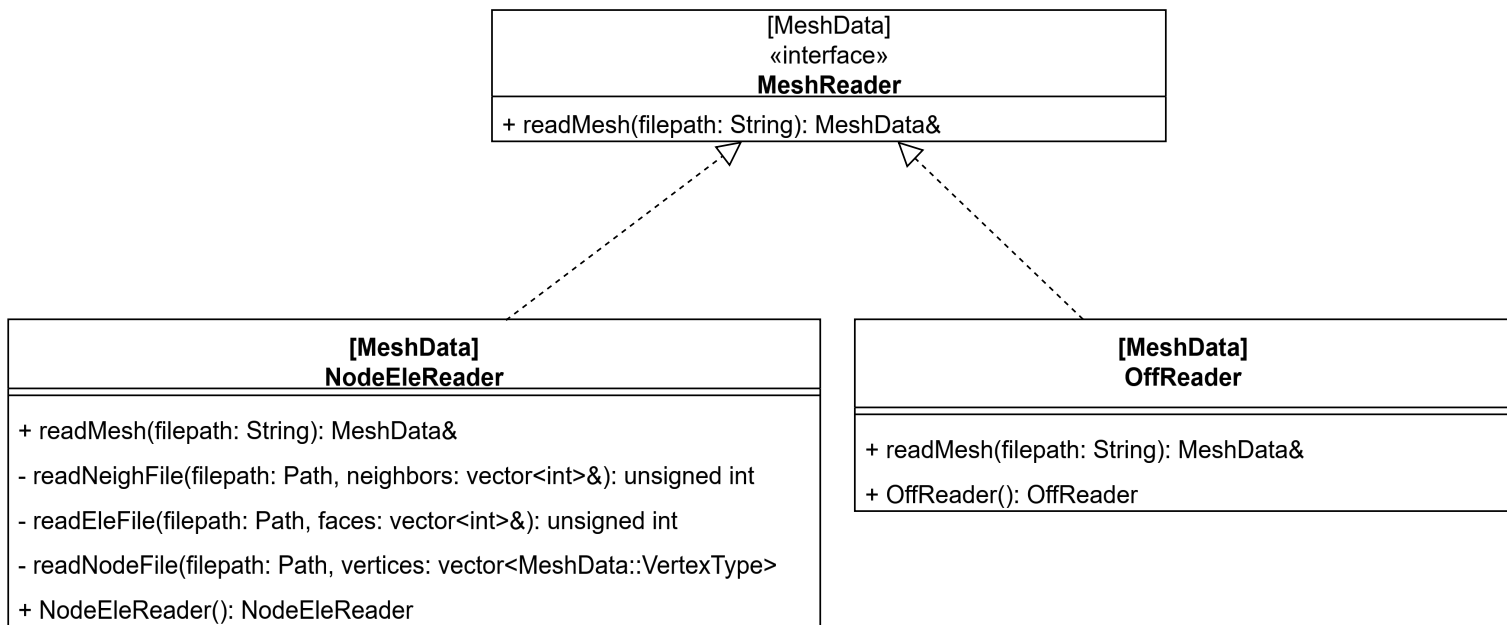


Figura 23: Interfaz y clases para leer la malla desde archivos

En cuanto al generador de mallas propuesto se hizo un desarrollo inicial de la clase **DelaunayCavityRefiner** (Figura 24), la cual sigue los pasos descritos en la solución propuesta, pero aún no está completa, ya que falta implementar la lógica de construcción de la nueva malla una vez que los triángulos que forman las cavidades son seleccionados (aquellos que contienen los puntos de los circuncentros de los triángulos elegidos por el criterio del refinador).

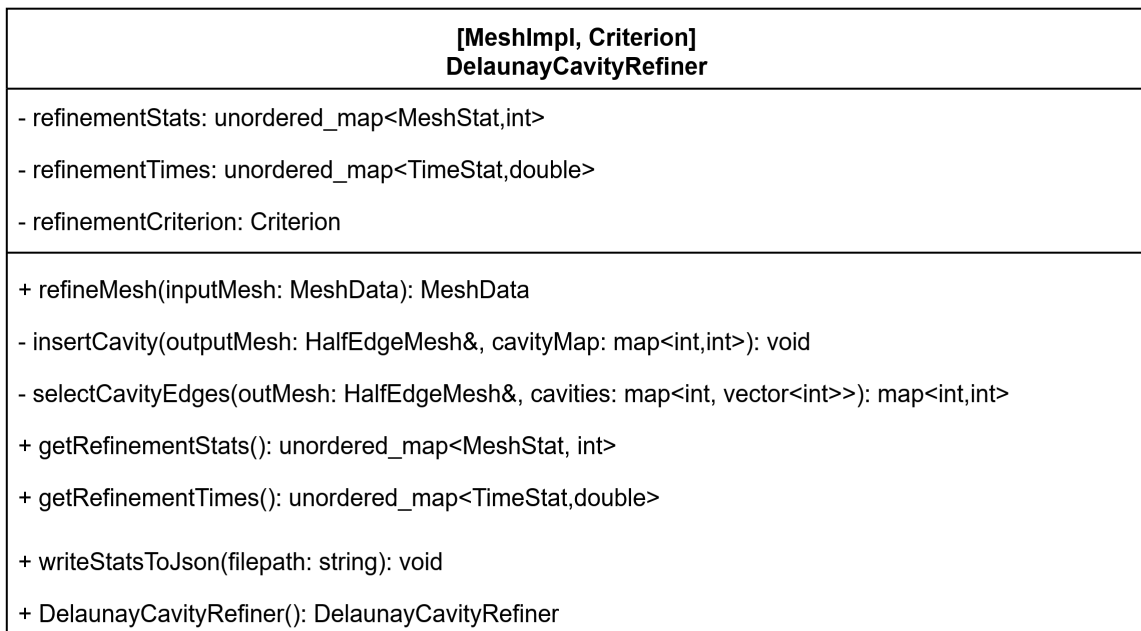


Figura 24: Clase para el refinador de mallas basado en cavidades

Para los criterios, se implementaron dos funtores básicos que permiten componer otros criterios, las estructuras `AndCriteria` y `OrCriteria` que cumplen la función de ser un «y» y un «o» lógico respectivamente. También se hizo la implementación inicial del funtor `MinAngleCriterion` (Figura 25), el cual toma como entrada una referencia a la malla, un índice de un triángulo y determina si el triángulo tiene un ángulo menor que el especificado en su atributo `angleThreshold`. La fórmula concreta para el cálculo de este ángulo mínimo se hace mediante la definición geométrica de la operación vectorial producto punto (definida como `dot` en la clase `Vertex`), la cual dice que

$$\cos \theta = \frac{A \cdot B}{\|A\| \|B\|}$$

$$\Rightarrow \cos^2 \theta = \frac{(A \cdot B)^2}{\|A\|^2 \|B\|^2}$$

Por lo que en el constructor, lo que realmente se guarda es el coseno cuadrado de `angleThreshold`, y luego este se compara con el coseno cuadrado del ángulo opuesto al lado más corto del triángulo, tal como se hace en el código del generador `Triangle` [20]. Esto hace el supuesto de que el ángulo a comparar es agudo, ya que después de $\frac{\pi}{2}$ el coseno cambia de signo y la comparación deja de ser útil. De igual manera todos los triángulos deben tener un lado más corto si no son equiláteros, y si es que lo son, ya son agudos.

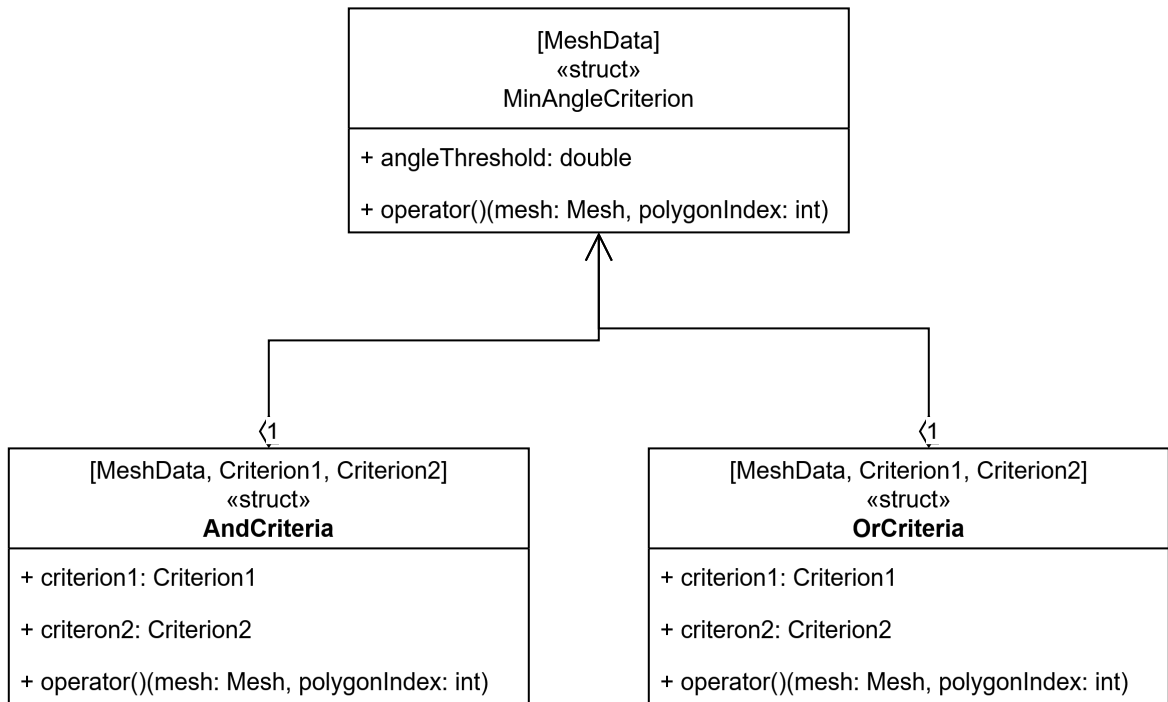


Figura 25: Funtores para componer criterios y funtor para criterio de ángulo mínimo

Estos avances están disponibles en el siguiente repositorio público iniciado como *fork* de Polylla[9]: <https://github.com/Tchy258/Delaunay-cavity>

Referencias

- [1] J. R. Shewchuk, «Triangle: Engineering a 2D quality mesh generator and Delaunay triangulator», en *Applied Computational Geometry Towards Geometric Engineering*, M. C. Lin y D. Manocha, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 1996, pp. 203-222.
- [2] H. Si, «Detri2 is a C++ program for generating (weighted) Delaunay triangulations for (weighted) point sets in 2d.». [En línea]. Disponible en: <https://www.wias-berlin.de/people/si/detri2.html>
- [3] B. Delaunay, «Sur la sphère vide. A la mémoire de Georges Voronoï», *Известия Российской академии наук. Серия математическая*, n.º 6, pp. 793-800, 1934.
- [4] S. Salinas-Fernández, N. Hitschfeld-Kahler, A. Ortiz-Bernardin, y H. Si, «POLYLLA: polygonal meshing algorithm based on terminal-edge regions», *Engineering with Computers*, vol. 38, n.º 5, pp. 4545-4567, 2022, doi: 10.1007/s00366-022-01643-4.
- [5] J. Ruppert, «A Delaunay Refinement Algorithm for Quality 2-Dimensional Mesh Generation», *Journal of Algorithms*, vol. 18, n.º 3, pp. 548-585, 1995, doi: <https://doi.org/10.1006/jagm.1995.1021>.
- [6] C. Lawson, «Software for C1 Surface Interpolation», *Mathematical Software*. Academic Press, pp. 161-194, 1977. doi: <https://doi.org/10.1016/B978-0-12-587260-7.50011-X>.
- [7] M.-C. Rivara, «New longest-edge algorithms for the refinement and/or improvement of unstructured triangulations», *International Journal for Numerical Methods in Engineering*, vol. 40, n.º 18, pp. 3313-3324, 1997, doi: [https://doi.org/10.1002/\(SICI\)1097-0207\(19970930\)40:18%3C3313::AID-NME214%3E3.0.CO;2-%23](https://doi.org/10.1002/(SICI)1097-0207(19970930)40:18%3C3313::AID-NME214%3E3.0.CO;2-%23).
- [8] G. Voronoi, «Nouvelles applications des paramètres continus à la théorie des formes quadratiques. Premier mémoire. Sur quelques propriétés des formes quadratiques positives parfaites.», *Journal für die reine und angewandte Mathematik (Crelles Journal)*, vol. 1908, n.º 133, pp. 97-102, 1908, doi: [doi:10.1515/crll.1908.133.97](https://doi.org/10.1515/crll.1908.133.97).
- [9] S. Salinas y R. Carrasco, *Polylla-Mesh-DCEL*. [En línea]. Disponible en: <https://github.com/ssalinasfe/Polylla-Mesh-DCEL/tree/main>

- [10] L. Veiga, F. Brezzi, A. Cangiani, G. Manzini, L. Marini, y A. Russo, «Basic Principles of Virtual Element Methods», *Mathematical Models and Methods in Applied Sciences*, vol. 23, pp. 199-214, 2012, doi: 10.1142/S0218202512500492.
- [11] S. Salinas y A. Chetan, *VEM-for-polylla-testing*. [En línea]. Disponible en: <https://github.com/ssalinasfe/VEM-for-polylla-testing>
- [12] A. Chetan, *VirtualElementMethods*. [En línea]. Disponible en: <https://github.com/justachetan/VirtualElementMethods>
- [13] H. Brönnimann, «Designing and Implementing a General Purpose Halfedge Data Structure», en *Algorithm Engineering*, G. S. Brodal, D. Frigioni, y A. Marchetti-Spaccamela, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 51-66.
- [14] K. J. Weiler, *Topological structures for geometric modeling (Boundary representation, manifold, radial edge structure)*. Rensselaer Polytechnic Institute, 1986.
- [15] J. R. Shewchuk, «Lecture Notes on Geometric Robustness», *Department of Electrical Engineering and Computer Sciences, University of California at Berkeley*, pp. 19-20, abr. 2013.
- [16] J. Richard Shewchuk, «Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates», *Discrete & Computational Geometry*, vol. 18, n.º 3, pp. 305-363, oct. 1997, doi: 10.1007/PL00009321.
- [17] R. C. Martin, «Design Principles and Design Patterns», 2000. [En línea]. Disponible en: https://web.archive.org/web/20150906155800/http://www.objectmentor.com/resources/articles/Principles_and_Patterns.pdf
- [18] «Constraints and concepts». [En línea]. Disponible en: <https://en.cppreference.com/w/cpp/language/constraints.html>
- [19] A. Sutton y B. Stroustrup, «Design of Concept Libraries for C++», 2011, pp. 97-118. doi: 10.1007/978-3-642-28830-2_6.
- [20] «Triangle A Two-Dimensional Quality Mesh Generator and Delaunay Triangulator.». [En línea]. Disponible en: <https://www.cs.cmu.edu/~quake/triangle.html>